

2520090240

Neha K

SKILL 4

```
mysql> USE sec_13;
Database changed
mysql> SELECT DATABASE();
+-----+
| DATABASE() |
+-----+
| sec_13      |
+-----+
1 row in set (0.00 sec)

mysql> CREATE TABLE class (
    ->     id INT,
    ->     name VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> CREATE TABLE class_info (
    ->     id INT,
    ->     address VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.05 sec)

mysql> SHOW TABLES;
+-----+
| Tables_in_sec_13 |
+-----+
| class             |
| class_info        |
| student           |
| student_marks1    |
+-----+
4 rows in set (0.00 sec)

mysql> INSERT INTO class_info VALUES
    -> (1, 'DELHI'),
    -> (2, 'MUMBAI'),
    -> (3, 'CHENNAI');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

CROSS JOIN

A CROSS JOIN combines every row of the first table with every row of the second table. It produces all possible combinations.

```
mysql> SELECT *
-> FROM class
-> CROSS JOIN class_info;
+-----+-----+-----+-----+
| id | name | id | address |
+-----+-----+-----+-----+
| 4 | alex | 1 | DELHI |
| 2 | adam | 1 | DELHI |
| 1 | abhi | 1 | DELHI |
| 4 | alex | 2 | MUMBAI |
| 2 | adam | 2 | MUMBAI |
| 1 | abhi | 2 | MUMBAI |
| 4 | alex | 3 | CHENNAI |
| 2 | adam | 3 | CHENNAI |
| 1 | abhi | 3 | CHENNAI |
+-----+-----+-----+-----+
9 rows in set (0.00 sec)
```

```
mysql> INSERT INTO class VALUES
-> (1, 'abhi'),
-> (2, 'adam'),
-> (3, 'alex'),
-> (4, 'anu');
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0
```

```
mysql>
mysql> INSERT INTO class_info VALUES
-> (1, 'DELHI'),
-> (2, 'MUMBAI'),
-> (3, 'CHENNAI');
Query OK, 3 rows affected (0.00 sec)
Records: 3 Duplicates: 0 Warnings: 0
```

INNER JOIN

An INNER JOIN returns only the records that have **matching values in both tables** based on the specified condition.

```
mysql> SELECT *  
      -> FROM class  
      -> INNER JOIN class_info  
      -> ON class.id = class_info.id;
```

id	name	id	address
1	abhi	1	DELHI
1	abhi	1	DELHI
2	adam	2	MUMBAI
2	adam	2	MUMBAI
1	abhi	1	DELHI
1	abhi	1	DELHI
2	adam	2	MUMBAI
2	adam	2	MUMBAI
3	alex	3	CHENNAI
3	alex	3	CHENNAI

10 rows in set (0.00 sec)

```
mysql> SELECT class.name,  
      ->      class_info.address  
      -> FROM class  
      -> INNER JOIN class_info  
      -> ON class.id=class_info.id;
```

name	address
abhi	DELHI
abhi	DELHI
adam	MUMBAI
adam	MUMBAI
abhi	DELHI
abhi	DELHI
adam	MUMBAI
adam	MUMBAI
alex	CHENNAI
alex	CHENNAI

10 rows in set (0.00 sec)

NATURAL JOIN

A NATURAL JOIN automatically joins tables using columns having the **same name**.

```
mysql> SELECT *  
      -> FROM class  
      -> NATURAL JOIN class_info;
```

id	name	address
1	abhi	DELHI
1	abhi	DELHI
2	adam	MUMBAI
2	adam	MUMBAI
1	abhi	DELHI
1	abhi	DELHI
2	adam	MUMBAI
2	adam	MUMBAI
3	alex	CHENNAI
3	alex	CHENNAI

```
10 rows in set (0.00 sec)
```

```
mysql> INSERT INTO class VALUES  
      -> (5, 'ashish');  
Query OK, 1 row affected (0.01 sec)
```

```
mysql>  
mysql> INSERT INTO class_info VALUES  
      -> (7, 'NOIDA'),  
      -> (8, 'PANIPAT');  
Query OK, 2 rows affected (0.01 sec)  
Records: 2  Duplicates: 0  Warnings: 0
```

LEFT OUTER JOIN

A LEFT JOIN returns all records from the left table and matching records from the right table. If there is no match, NULL values are returned.

```
mysql> SELECT *
-> FROM class
-> LEFT OUTER JOIN class_info
-> ON class.id=class_info.id;
```

id	name	id	address
1	abhi	1	DELHI
1	abhi	1	DELHI
2	adam	2	MUMBAI
2	adam	2	MUMBAI
4	alex	NULL	NULL
1	abhi	1	DELHI
1	abhi	1	DELHI
2	adam	2	MUMBAI
2	adam	2	MUMBAI
3	alex	3	CHENNAI
3	alex	3	CHENNAI
4	anu	NULL	NULL
5	ashish	NULL	NULL

13 rows in set (0.00 sec)

```
mysql>
mysql> SELECT *
-> FROM class
-> LEFT JOIN class_info
-> ON class.id=class_info.id
-> WHERE class_info.id IS NULL;
```

id	name	id	address
4	alex	NULL	NULL
4	anu	NULL	NULL
5	ashish	NULL	NULL

3 rows in set (0.00 sec)

RIGHT OUTER JOIN

A RIGHT JOIN returns all records from the right table and matching records from the left table. If there is no match, NULL values are returned.

```
mysql> SELECT *  
      -> FROM class  
      -> RIGHT OUTER JOIN class_info  
      -> ON class.id=class_info.id;
```

id	name	id	address
1	abhi	1	DELHI
1	abhi	1	DELHI
2	adam	2	MUMBAI
2	adam	2	MUMBAI
3	alex	3	CHENNAI
1	abhi	1	DELHI
1	abhi	1	DELHI
2	adam	2	MUMBAI
2	adam	2	MUMBAI
3	alex	3	CHENNAI
NULL	NULL	7	NOIDA
NULL	NULL	8	PANIPAT

12 rows in set (0.00 sec)

```
mysql>  
mysql> SELECT *  
      -> FROM class  
      -> RIGHT JOIN class_info  
      -> ON class.id=class_info.id  
      -> WHERE class.id IS NULL;
```

id	name	id	address
NULL	NULL	7	NOIDA
NULL	NULL	8	PANIPAT

2 rows in set (0.00 sec)

FULL OUTER JOIN

A FULL OUTER JOIN returns all records from both tables, including matched and unmatched records.

```
mysql> SELECT *  
      -> FROM class  
      -> LEFT JOIN class_info  
      -> ON class.id = class_info.id  
      ->  
      -> UNION  
      ->  
      -> SELECT *  
      -> FROM class  
      -> RIGHT JOIN class_info  
      -> ON class.id = class_info.id;
```

id	name	id	address
1	abhi	1	DELHI
2	adam	2	MUMBAI
4	alex	NULL	NULL
3	alex	3	CHENNAI
4	anu	NULL	NULL
5	ashish	NULL	NULL
NULL	NULL	7	NOIDA
NULL	NULL	8	PANIPAT

8 rows in set (0.01 sec)

```
mysql> SELECT *
-> FROM class
-> LEFT JOIN class_info
-> ON class.id = class_info.id
-> WHERE class_info.id IS NULL
->
-> UNION
->
-> SELECT *
-> FROM class
-> RIGHT JOIN class_info
-> ON class.id = class_info.id
-> WHERE class.id IS NULL;
```

id	name	id	address
4	alex	NULL	NULL
4	anu	NULL	NULL
5	ashish	NULL	NULL
NULL	NULL	7	NOIDA
NULL	NULL	8	PANIPAT

5 rows in set (0.00 sec)


```
mysql> CREATE TABLE first_table(  
  -> id INT,  
  -> name VARCHAR(30)  
  -> );  
Query OK, 0 rows affected (0.04 sec)  
  
mysql>  
mysql> CREATE TABLE second_table(  
  -> id INT,  
  -> name VARCHAR(30)  
  -> );  
Query OK, 0 rows affected (0.04 sec)  
  
mysql>  
mysql> INSERT INTO first_table VALUES  
  -> (1, 'abhi'),  
  -> (2, 'adam');  
Query OK, 2 rows affected (0.01 sec)  
Records: 2  Duplicates: 0  Warnings: 0  
  
mysql>  
mysql> INSERT INTO second_table VALUES  
  -> (2, 'adam'),  
  -> (3, 'chester');  
Query OK, 2 rows affected (0.01 sec)  
Records: 2  Duplicates: 0  Warnings: 0
```

Set Operations

UNION

Combines the results of two queries and **removes duplicate records**.

```
mysql> SELECT * FROM first_table  
-> UNION  
-> SELECT * FROM second_table;
```

id	name
1	abhi
2	adam
3	chester

3 rows in set (0.00 sec)

```
mysql>  
mysql> SELECT name FROM first_table  
-> UNION  
-> SELECT name FROM second_table;
```

name
abhi
adam
chester

3 rows in set (0.00 sec)

```
mysql> SELECT * FROM first_table  
-> UNION ALL  
-> SELECT * FROM second_table;
```

id	name
1	abhi
2	adam
2	adam
3	chester

4 rows in set (0.00 sec)

UNION ALL

Combines the results of two queries and **keeps duplicate records**.

```
mysql> SELECT COUNT(*)  
-> FROM  
-> (  
-> SELECT * FROM first_table  
-> UNION ALL  
-> SELECT * FROM second_table  
-> ) A;
```

COUNT(*)
4

1 row in set (0.01 sec)

```
mysql> SELECT * FROM first_table  
-> INTERSECT  
-> SELECT * FROM second_table;
```

id	name
2	adam

1 row in set (0.00 sec)

```
mysql> SELECT name FROM first_table  
-> INTERSECT  
-> SELECT name FROM second_table;
```

name
adam

1 row in set (0.00 sec)

```
mysql> SELECT f.*  
-> FROM first_table AS f  
-> LEFT JOIN second_table AS s  
-> ON f.id = s.id  
-> WHERE s.id IS NULL;
```

id	name
1	abhi

1 row in set (0.00 sec)

```
mysql> SELECT name  
-> FROM first_table  
-> WHERE name NOT IN (  
->     SELECT name  
->     FROM second_table  
-> );
```

name
abhi

1 row in set (0.00 sec)

```
mysql> SELECT c.id,c.name,ci.address
-> FROM class c
-> INNER JOIN class_info ci
-> ON c.id=ci.id;
```

id	name	address
1	abhi	DELHI
1	abhi	DELHI
2	adam	MUMBAI
2	adam	MUMBAI
3	alex	CHENNAI
1	abhi	DELHI
1	abhi	DELHI
2	adam	MUMBAI
2	adam	MUMBAI
3	alex	CHENNAI

10 rows in set (0.00 sec)

```
mysql>
mysql> SELECT c.id,
-> c.name,
-> CASE
-> WHEN ci.address IS NULL
-> THEN 'Address Missing'
-> ELSE 'Address Available'
-> END AS Status
-> FROM class c
-> LEFT JOIN class_info ci
-> ON c.id=ci.id;
```

id	name	Status
1	abhi	Address Available
1	abhi	Address Available
2	adam	Address Available
2	adam	Address Available
4	alex	Address Missing
1	abhi	Address Available
1	abhi	Address Available
2	adam	Address Available
2	adam	Address Available
3	alex	Address Available
3	alex	Address Available
4	anu	Address Missing
5	ashish	Address Missing

13 rows in set (0.00 sec)

- INNER JOIN → Matching records only
- LEFT JOIN → All left + matching right
- RIGHT JOIN → All right + matching left
- FULL JOIN → Everything from both
- CROSS JOIN → Every possible combination
- UNION → Combine, remove duplicates
- UNION ALL → Combine, keep duplicates
- INTERSECT → Common records
- MINUS → Only in first result